

Power-of-Two Quantization-Aware-Training (PoT-QAT) in Large Language Models (LLMs)

Mahmoud Elgenedy

Department of Computer Science, Stanford University
melgened@stanford.edu

Abstract

In Large Language Models (LLMs), the number of parameters has grown exponentially in the past few years, e.g., from 1.5 billion parameters in GPT-2 to 175 billion in GPT-3 to possibly more than trillion in higher versions. This raises a significant challenge for implementation, especially for Edge devices. Unlike cloud computing, memory and processing power for Edge devices are very limited, which necessitates developing novel ideas to make such applications feasible. In this work, we investigate compressing weights with a special quantization that limits numbers to only power-of-two (PoT). This helps save a huge amount of memory as only exponents need to be stored, more importantly, it significantly reduces processing power by replacing costly multiplication with low cost bit shifting. To overcome performance loss due to this strict quantization, we investigate Quantization Aware Training (QAT) to enhance performance through additional training. Results on GPT-2 124M show a major enhancement for quantized PoT model after additional training, with a perplexity enhancement of 66% and BERT-Score loss to baseline GPT-2 of 1%. The memory saving is estimated to be 87.5% while the inference speed is expected to be 3-10x faster with PoT quantization versus full-precision.

1 Introduction

Quantization is a very effective method to optimize and reduce Deep Neural Networks DNN complexity, where numbers are represented with lower precision to help reduce both memory and processing requirements [1]. In this proposal, for the LLM models, we investigate restricting weights to power-of-two (PoT), aiming at a significant reduction in memory and computations. Moreover, to enhance the performance, we extend our work in [2] by exploring Quantization Aware Training (QAT). For other quantization approaches for LLM with QAT, refer to [3].

While quantization in neural networks is widely researched [4-6], only few studies investigated power-of-two quantization [7-10]. Specifically, restricting weights to PoT is investigated in a couple of studies for general DNNs [7-8] and with some focus on image applications using Convolutional Neural Networks (CNNs) [9]. However, for LLMs applications, very limited researches are available including a recent publication [10] that proposes PoT with post training quantization (PTQ) similar to the approach we proposed in our earlier study in [2].

2 Dataset and Model

NanoGPT [11] is a simple and fast repository for training and fine tuning Generative Pre-trained Transformer GPT. The design follows the GPT-2 model and can reproduce GPT-2 on OpenWebText dataset. Moreover, it supports various settings, allowing fast trials and smaller datasets, including Shakespeare dataset. For preliminary results, we consider the Shakespeare dataset, partitioned as 80%-10%-10% for training-val-testing, and focuses mainly on character-based model with model/parameters shown in figure 1. This is to help with short turn around testing and fine tuning, with more focus on the algorithm and correct implementation of the QAT for PoT. For more realistic results, we extended our PoT-QAT results to GPT-2 model. The baseline model is the 124M parameters pretrained model [12]. The QAT is fine tuned using same dataset used for GPT-2 training, OpenWebText dataset [13].

3 Quantization Aware Training

There are two different approaches for DNNs quantization: Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT). In PTQ, training is done completely in floating point high precision, then weights and activations are quantized into lower precision during inference, no further tuning after quantization. PTQ is less complex but suffers higher performance loss.

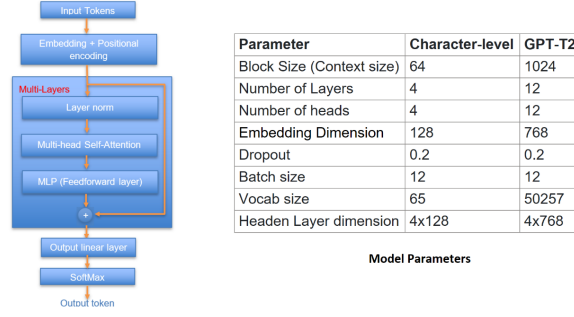


Figure 1: LLM Model illustration and parameters

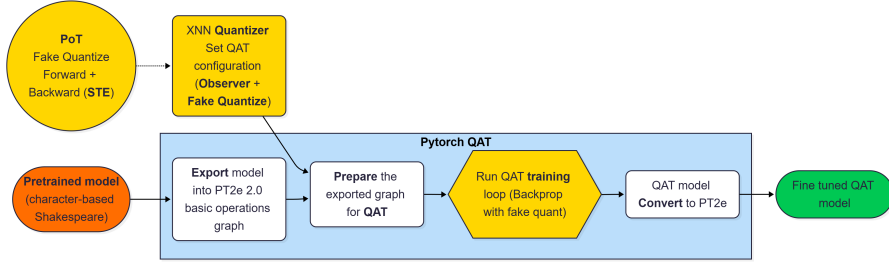


Figure 2: PyTorch QAT Flow

On the other hand, in QAT, quantization is part of the training process, where fake quantize modules (that simulate the quantization loss by quantizing numbers followed by dequantizing/rescaling to floating range) are inserted between different operations, allowing training to fine tune the parameters considering quantization loss. Note that some of those operations like rounding are not differentiable which is a challenge to compute gradients for backward propagation. A common trick used is called straight-through estimator (STE) which treats the steps of rounding as identity only in the backward path, allowing gradients to flow through. STE approximation is proven to work well in practice and enables the model to converge and adapt to quantization noise during training [14].

QAT is computationally more expensive than PTQ but very effective to recover degradation due to quantization using additional training. Moreover, QAT is very critical when lower bit-widths like INT4 are used, or when models are sensitive to quantization noise, such as CNNs or transformers. Since we are examining here a very aggressive quantization technique (PoT), we believe QAT would be a very effective approach to enhance the large performance degradation observed for PTQ [2].

We use most recent PyTorch 2 Export Mode quantization framework [15-16] as it is more flexible and easily scalable. As in figure 2, the flow of QAT in PyTorch starts with exporting the model into a graph of basic operations. Next in prepare stage, fake quantizer modules are inserted between each two operations, based on configuration provided through the quantizer block. Inside quantizer we can configure fake quantize operation, quantization type and bit widths, in addition to signal statistics. In particular, the so called observer block is used to depict signal statistics (like min and max), then fake quantize can use these statistics to scale and quantize/dequantize signals. After prepare, the model is now ready for training to tune the weights with quantization effect. Lastly, after training is done, a convert process is used to remove observers and only keep fake quantization modules for inference.

4 Power of Two Quantization

Power-of-two quantization is an aggressive quantization approach that limits quantized integer numbers to be only power of two. Multiplication (which is essential part of element-wise, matrix multiplication or convolution) is a major part of any DNN processing complexity. When one of the arguments has numbers that are restricted to be PoT, the multiplication operation turns into simple bit shifting which significantly reduces the computational complexity. Table 1 shows the formulas of the PoT versus Uniform affine quantization. The drawback of the PoT is the expected performance loss, which is the main motivation to introduce QAT trying to overcome that loss.

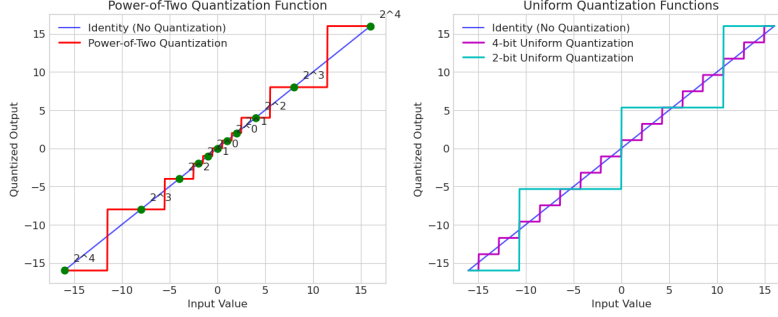


Figure 3: Power-of-Two (PoT) Versus Uniform Quantization

Table 1: Types of Quantization

Quantization type	Conversion equation (input x, and output y)	Use case
Asymmetric (Affine)	$y = \text{round}(x/\text{scale}) + zp$	Activations
Power of Two (PoT)	$y = 2^{\text{clip}(\text{round}(\log_2(x/\text{scale})))}$	PoT Weights

As in uniform quantization, operation of PoT is not differentiable, and hence we also need to approximate the PoT into differentiable operation in the backward path. We examined the STE approach which seems to be really a good approximation and based on initial experiments, LLM model is able to converge. In figure 3, we show the quantization output of both PoT and uniform quantization. PoT is a logarithmic approach where steps are not uniform (this can be beneficial for large outliers as a side benefit of PoT). STE assumes identity function in the backward path, i.e., output is same as input (the blue straight line in the figure).

5 Experiments and Performance Evaluation

5.1 Character based model with Shakespeare dataset

For initial trials, we investigated the QAT performance of the character based LLM, where weights are quantized as uniform vs PoT (note that to benefit from PoT, only weights are needed to be in PoT). In table 2 we show training/validation loss convergence during training. We first trained the model on floating point for 15000 iterations with $LR = 10^{-3}$ which lowered cross entropy loss from little above 4 to 1.6. We then load the trained model inside the QAT and train ($LR = .5 * 10^{-5}$) for both uniform and PoT quantization. The table shows results for uniform quantization 4-bits [-8, 7], versus 9-levels PoT. The results show clear convergence for both techniques, and proves that PoT can be enhanced with QAT. Specifically, the performance suffers an initial drop once applying quantization (from 1.6 baseline to above 1.8 loss), then converges back after some training iterations.

5.2 GPT-2 124M model with openwebtext data-set

We extended results of the PoT-QAT on the GPT-2 124M model. We use different metrics to analyze performance, including train/val loss convergence curves, perplexity, and BERT-Score.

5.2.1 Training/Validation loss convergence

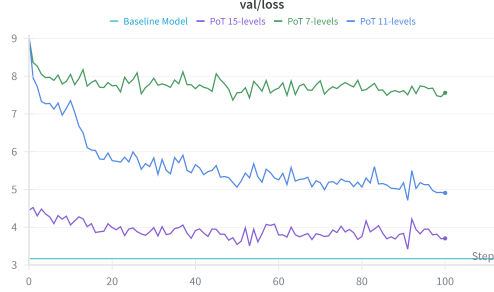
As shown in figures 4a and 4b, the training and validation loss successfully converges. We tried different values of learning rates LR s and $.5e - 5$ was the successful pick. The context length is 64, while the batch size is chosen to be 12 (we swept over different batch sizes 12, 24, 48, 64). We tried different resolution of PoT quantization (7, 11 and 15 levels). PoT with 15 levels ($[-2^7, 2^7]$) is

Table 2: Nano-GPT Character Based LLM QAT Performance. Uniform quantization versus PoT

Iteration	Uniform Quantization		Power-of-Two	
	Training	Validation	Training	Validation
15500	1.61635	1.84743	1.65916	1.82383
19500	1.56602	1.68807	1.63777	1.75756
22500	1.61594	1.59317	1.66232	1.68218



(a)



(b)

Figure 4: Quantization Aware Training convergence for different PoT settings. Each step is 5 training step iterations. (a) Training loss versus iterations. (b) Validation loss versus iterations.

Table 3: Perplexity performance of PoT post training quantization PTQ versus PoT quantization aware training quantization QAT

quantization range		PoT - PTQ		PoT - QAT	
Integer range	N bits	l_{ce}	Perplexity	l_{ce}	Perplexity
Baseline float32	32 bits	3.167	23.73	3.167	23.167
$[-2^3, 2^3]$	3 bits	9.01	8184	7.42	1669
$[-2^5, 2^5]$	4 bits	8.76	6374	4.71	111
$[-2^7, 2^7]$	4 bits	4.5	90	3.417	30.47

able to close the performance gap degradation compared to baseline model. Compared to other PoT variants, PoT 15 levels shows initial small bias, while a slight overfitting is observed toward end of training. On the other hand, PoT with 7 and 11 levels show relatively large bias which is enhanced with training especially for the 11 levels (further training for 11 levels may gain more enhancement).

5.2.2 Perplexity

Perplexity is a measure of how accurate the model can predict next token. In other words, it measures how the model is able to learn the data distribution. As the name implies, lower perplexity means model is able to predict with less ambiguity. It is calculated as the exponential of the cross entropy loss l_{ce} , i.e., perplexity is $\exp(l_{ce})$

To measure cross-entropy, we average over multiple iterations of batches captured from val set. The cross-entropy $l_{ce}(t, y)$ with input k -dimension logits $t \in \mathbb{R}^k$ and target class y is defined as, $l_{ce}(t, y) = -\log P(y; t) = -\log \left(\frac{\exp(t_y)}{\sum_s \exp(t_s)} \right)$

In Table 3, we show the perplexity of quantized PoT for Post training PTQ, i.e., PoT quantization without additional training (prior results) versus the introduced quantization aware training QAT. The results show a major enhancement with training (QAT). For example, for the 15-levels PoT (equivalently 4 bits), the QAT is able to lower the perplexity from 90 (of PTQ) to 30, which is a significant enhancement (over 66%), and is very close to baseline model perplexity of 23.7.

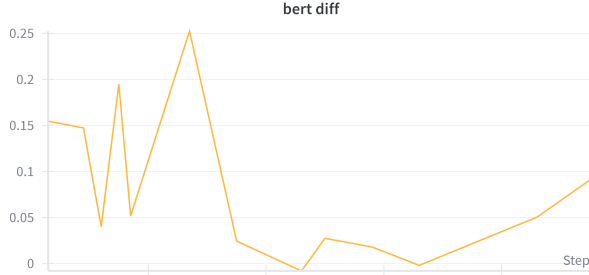


Figure 5: BERT-Score delta for PoT-QAT quantized vs baseline floating-point, at val loss ≈ 3.78

5.2.3 BERT-Score

BERT-Score is a similarity score for each token in the candidate sequence with each token in the reference sequence, where token similarity is done using contextual embeddings instead of exact matching [17]. BERT-Score is a good indication to human judgment. We use Hugging Face evaluate package to calculate the BERT-Score, with base model "roberta-base". We average over 10 iterations, where for each iteration we generate a new text and compute the BERT-Score. In figure 5, we plot the BERT-Score difference between baseline floating point and quantized model with training iterations, only reported at checkpoints where better validation is observed. The BERT-Score is computed for baseline versus reference data, and similarly for the quantized model. The average BERT-Score of the baseline model is about 0.2, which is not very high, but expected considering GPT-2 124M. It is clear from the curve that the gap between quantized and baseline GPT-2 is becoming closer with training progressing, until reaching smallest BERT-Score difference (0.0019 difference $\approx 1\%$ loss) around validation iteration 66 (training step 330). It worth noting that best BERT-Score is not at best validation error (best BERT-Score is aligned with validation error ≈ 3.78 while best validation error is 3.41), which may indicate some overfitting toward the end of the training loop. This is very useful to pick the best model, where all metrics should be considered. For reference we show in figures 7a and 7b in the appendix a sample of generated text by both base and quantized models.

6 Memory and Computational Savings

Table 4, shows a summary of the memory and computation savings of using 4-bits PoT quantization. The memory saving is an outcome of using smaller bit-width (4-bits in PoT versus 32-bits of baseline float32), which is similar to any 4-bits quantization technique. However, the computation saving is a combination of saving from using smaller bit-width weights in all matrix multiplications (2-5x faster) in addition to the bit shifting operation that replaces multiplication (additional 1.5-2x faster).

Table 4: GPT-2 124M: FP32 vs. 4-bit Power-of-Two Quantization

Metric	FP32	4-bit PoT
Model Size	124 MB x 4 = 496 MB	62 MB (87.5% saving)
Inference Speed	1x	3-10x faster

7 Conclusions and future work

We proposed power of two quantization for LLM weights which significantly reduces memory and computational complexity. We proposed using QAT so that additional training can help mitigate the performance degradation due to aggressive quantization. PoT functionality is implemented within PyTorch QAT framework for nano-GPT code. Results for GPT-2 124M show significant enhancement for the PoT with QAT compared to quantization without training (PTQ). Perplexity for 15 levels PoT gained 66% enhancement over PTQ. The BERT-Score is also showing clear enhancement with training loop, where difference of BERT-Score versus baseline model approaches 1%. Future works includes experimenting the PoT-QAT for bigger models like Llama and Qwen. Also, it will be very interesting to try the algorithm on device and measure the actual complexity saving.

Contributions

- Implementing the QAT framework for nano-GPT LLM model, and configure/optimize model parameters (like LR) and quantization parameters to achieve convergence.
- Implement The PoT differentiable class that Includes both forward and backward functions.
- Integrating the PoT class inside the QAT framework (inheriting the fake quantize class).
- Achieving convergence for QAT for both uniform and PoT on the character level LLM model as first step.
- Extended our results for more realistic GPT-2 124M parameters. Achieved convergence for the PoT-QAT for different versions of PoT with different resolution.
- Measured the performance using different metrics including train/val loss, perplexity, and BERT-Score.
- Finally, performance metrics show a great success for the QAT to help PoT to close the degradation gap compared to the baseline model (66% enhancement for perplexity over PTQ and 1% loss of BERT-Score versus full precision)

References

- [1] What are Quantized LLMs? "<https://www.tensorops.ai/post/what-are-quantized-llms#viewer-2tmd3>"
- [2] Elgenedy, Mahmoud. "Power-of-Two (PoT) Weights in Large Language Models (LLMs)." arXiv preprint arXiv:2506.00315 (2025).
- [3] Liu, Zechun, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. "Llm-qat: Data-free quantization aware training for large language models." In Findings of the Association for Computational Linguistics: ACL 2024, pp. 467-484. 2024.
- [4] Gholami, Amir, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. "A survey of quantization methods for efficient neural network inference." In Low-Power Computer Vision, pp. 291-326. Chapman and Hall/CRC, 2022.
- [5] Krishnamoorthi, Raghuraman. "Quantizing deep convolutional networks for efficient inference: A whitepaper." arXiv preprint arXiv:1806.08342 (2018).
- [6] Wu, Hao, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. "Integer quantization for deep learning inference: Principles and empirical evaluation." arXiv preprint arXiv:2004.09602 (2020).
- [7] Elhoushi, Mostafa, Zihao Chen, Farhan Shafiq, Ye Henry Tian, and Joey Yiwei Li. "Deepshift: Towards multiplication-less neural networks." In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 2359-2368. 2021.
- [8] Przewlocka-Rus, Dominika, Syed Shakib Sarwar, H. Ekin Sumbul, Yuecheng Li, and Barbara De Salvo. "Power-of-two quantization for low bitwidth and hardware compliant neural networks." arXiv preprint arXiv:2203.05025 (2022).
- [9] McDanel, Bradley, Sai Qian Zhang, H. T. Kung, and Xin Dong. "Full-stack optimization for accelerating cnns using powers-of-two weights with fpga validation." In Proceedings of the ACM International Conference on Supercomputing, pp. 449-460. 2019.
- [10] Wang, Xinyu, Vahid Partovi Nia, Peng Lu, Jerry Huang, Xiao-Wen Chang, Boxing Chen, and Yufei Cui. "PoTPTQ: A Two-step Power-of-Two Post-training for LLMs." arXiv preprint arXiv:2507.11959 (2025).
- [11] <https://github.com/karpathy/nanoGPT>, by Karpathy
- [12] <https://huggingface.co/openai-community/gpt2>
- [13] <https://huggingface.co/datasets/Skylion007/openwebtext>
- [14] <https://api.wandb.ai/links/byyoung3/eb1e1lcu>
- [15] <https://pytorch.org/docs/stable/quantization.html#quantization-api-summary>
- [16] (prototype) PyTorch 2.0 Export Post Training Static Quantization — PyTorch Tutorials 2.0.1+cu117 documentation
- [17] Zhang, Tianyi, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. "Bertscore: Evaluating text generation with bert." arXiv preprint arXiv:1904.09675 (2019).

8 Acknowledgments

I have done all work myself, Mahmoud Elgenedy, and no other team members. I would like to acknowledge my Instructors and TAs of CS230 for all their help and support. The Latex template is slightly modified from, "CS230: Deep Learning, Winter, 2018, Stanford University, CA. (LateX template borrowed from NIPS 2017.)" I got some help in basics questions from internet search and using LLMs.

Appendix

8.1 Codes

All codes are uploaded to following github,

https://github.com/melgenedy/Quantized_nano_GPT-QAT-POT.git

8.2 Some Snippets from Quantized model

opcode	name	target	args	kwargs
get_attr	transformer_wpe_weight	transformer.wpe.weight	()	()
get_attr	transformer_h_0_in_1_weight	transformer.h.0.in_1.weight	()	()
get_attr	_frozen_param0	_frozen_param0	()	()
call_function	dequantize_per_tensor_default	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param0, 0.06372686475515366, 0, -8, 7, torch.int8)	()
get_attr	_frozen_param1	_frozen_param1	()	()
call_function	dequantize_per_tensor_default_1	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param1, 0.016682995483279228, 0, -8, 7, torch.int8)	()
get_attr	transformer_h_0_in_2_weight	transformer.h.0.in_2.weight	()	()
get_attr	_frozen_param2	_frozen_param2	()	()
call_function	dequantize_per_tensor_default_2	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param2, 0.043351493775844574, 0, -8, 7, torch.int8)	()
get_attr	_frozen_param3	_frozen_param3	()	()
call_function	dequantize_per_tensor_default_3	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param3, 0.04125349968671799, 0, -8, 7, torch.int8)	()
get_attr	transformer_h_1_in_1_weight	transformer.h.1.in_1.weight	()	()
get_attr	_frozen_param4	_frozen_param4	()	()
call_function	dequantize_per_tensor_default_4	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param4, 0.03207191079854965, 0, -8, 7, torch.int8)	()
get_attr	_frozen_param5	_frozen_param5	()	()
call_function	dequantize_per_tensor_default_5	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param5, 0.021004871459174156, 0, -8, 7, torch.int8)	()
get_attr	transformer_h_1_in_2_weight	transformer.h.1.in_2.weight	()	()
get_attr	_frozen_param6	_frozen_param6	()	()
call_function	dequantize_per_tensor_default_6	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param6, 0.03638213439464569, 0, -8, 7, torch.int8)	()
get_attr	_frozen_param7	_frozen_param7	()	()
call_function	dequantize_per_tensor_default_7	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param7, 0.0440548397606598, 0, -8, 7, torch.int8)	()
get_attr	transformer_h_2_in_1_weight	transformer.h.2.in_1.weight	()	()
get_attr	_frozen_param8	_frozen_param8	()	()
call_function	dequantize_per_tensor_default_8	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param8, 0.0368971973657608, 0, -8, 7, torch.int8)	()
get_attr	_frozen_param9	_frozen_param9	()	()
call_function	dequantize_per_tensor_default_9	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param9, 0.0228872112929821, 0, -8, 7, torch.int8)	()
get_attr	transformer_h_2_in_2_weight	transformer.h.2.in_2.weight	()	()
get_attr	_frozen_param10	_frozen_param10	()	()
call_function	dequantize_per_tensor_default_10	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param10, 0.03949552774429321, 0, -8, 7, torch.int8)	()
get_attr	_frozen_param11	_frozen_param11	()	()
call_function	dequantize_per_tensor_default_11	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param11, 0.048228416591882786, 0, -8, 7, torch.int8)	()
get_attr	transformer_h_3_in_1_weight	transformer.h.3.in_1.weight	()	()
get_attr	_frozen_param12	_frozen_param12	()	()
call_function	dequantize_per_tensor_default_12	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param12, 0.03666684408015541, 0, -8, 7, torch.int8)	()
get_attr	_frozen_param13	_frozen_param13	()	()
call_function	dequantize_per_tensor_default_13	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param13, 0.030389339985251427, 0, -8, 7, torch.int8)	()
get_attr	transformer_h_3_in_2_weight	transformer.h.3.in_2.weight	()	()
get_attr	_frozen_param14	_frozen_param14	()	()
call_function	dequantize_per_tensor_default_14	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param14, 0.043360400944948196, 0, -8, 7, torch.int8)	()
get_attr	_frozen_param15	_frozen_param15	()	()
call_function	dequantize_per_tensor_default_15	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param15, 0.09454374815331268, 0, -8, 7, torch.int8)	()
get_attr	transformer_ln_f_weight	transformer.ln_f.weight	()	()
get_attr	lm_head_weight	lm_head.weight	()	()
get_attr	_frozen_param16	_frozen_param16	()	()
call_function	dequantize_per_tensor_default_16	quantized_decomposed.dequantize_per_tensor.default	(_frozen_param16, 0.08917881548404694, 0, -8, 7, torch.int8)	()
get_attr	lifted_tensor_0	lifted_tensor_0	()	()
placeholder	idx	idx	()	()
call_function	arange	aten.arange.start	(0, 64)	()
call_function	embedding	aten.embedding.default	(lm_head_weight, idx)	()
call_function	quantize_per_tensor_default_17	quantized_decomposed.quantize_per_tensor.default	(embedding, 0.002691424684599042, -1, -128, 127, torch.int8)	()
call_function	dequantize_per_tensor_default_17	quantized_decomposed.dequantize_per_tensor.default	(quantize_per_tensor_default_17, 0.002691424684599042, -1, -128, 127, torch.int8)	()
call_function	embedding_1	aten.embedding.default	(transformer_wpe_weight, arange)	()
call_function	quantize_per_tensor_default_18	quantized_decomposed.quantize_per_tensor.default	(embedding_1, 0.0042237816378474236, 68, -128, 127, torch.int8)	()
call_function	dequantize_per_tensor_default_18	quantized_decomposed.dequantize_per_tensor.default	(quantize_per_tensor_default_18, 0.0042237816378474236, 68, -128, 127, torch.int8)	()
call_function	add	aten.add.Tensor	(dequantize_per_tensor_default_17, dequantize_per_tensor_default_18)	()
call_function	quantize_per_tensor_default_19	quantized_decomposed.quantize_per_tensor.default	(add, 0.00507753249257803, 43, -128, 127, torch.int8)	()
call_function	dropout_558	quantized_decomposed.dequantize_per_tensor.default	(quantize_per_tensor_default_19, 0.00507753249257803, 43, -128, 127, torch.int8)	()
call_function	quantize_per_tensor_default_20	aten.dropout.default	(dropout_558, 0.008676772005856037, 45, -128, 127, torch.int8)	()
call_function	dequantize_per_tensor_default_20	quantized_decomposed.dequantize_per_tensor.default	(quantize_per_tensor_default_20, 0.008676772005856037, 45, -128, 127, torch.int8)	()
call_function	layer_norm	aten.layer_norm.default	(dropout_558, [128], transformer_h_0_in_1_weight)	()
call_function	quantize_per_tensor_default_21	quantized_decomposed.quantize_per_tensor.default	(layer_norm, 0.0102182327720563, 18, -128, 127, torch.int8)	()

Figure 6: A snippet from the quantized model shown as print tabular

8.3 Some Snippets from generated Quantized model versus baseline GPT-2 model

```
print(decode(y_org[0].tolist()),end="")
. There are formulations of quantum mechanics where collapse of the wave function is truly random and unpredictable; there are others with
hidden variables, in which the true state of the universe isn't defined by the wave function. In any of those cases, our analysis is comple
tely beside the point. It's interesting to note that the best single-variable analysis of the state of the universe is by applying a discre
te discrete state property. And that is what makes it in our view relevant to our discussion. But what is important is to understand why th
at is so.

Why? Well the evidence suggests that because space is finite, there
```

(a)

```
print(decode(y_last[0,-1].tolist()),end="")
. There are formulations of quantum mechanics where collapse of the wave function is truly random and unpredictable; there are others with
hidden variables, in which the true state of the universe isn't defined by the wave function. In any of those cases, our analysis is comple
tely beside the point. It's interesting to note that quantum mechanics will be able to explain so much of the cosmological system in such a
way's natural system's laws of space, in particular, which to illustrate these phenomena in sequence's complexity, as reflected in these gr
avitational forces. In light of these equations, these theories
```

(b)

Figure 7: Generated text with context length 64 (highlighted), temperature = 0.8. The generated text from quantized model at validation loss 3.76 seems very plausible compared to the baseline model. Note that baseline model is GPT-2 124M, which is relatively a small model compared to newer or larger parameters model, so the generated text even from base model is limited but is very useful to proof of concept. (a) Baseline GPT-2 124M parameters, floating point. (b) PoT-QAT quantized model at val loss ≈ 3.7 .